

Rockchip RK3326 Linux SDK 快速入门

文件标识：RK-JC-YF-943

发布版本：V2.1.0

日期：2023-12-20

文件密级：☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2023 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：福建省福州市铜盘路软件园A区18号

网址：www.rock-chips.com

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：fae@rock-chips.com

前言

概述

本文主要描述了 RK3326 Linux SDK 的基本使用方法，旨在帮助开发者快速了解并使用 RK3326 Linux SDK 开发包。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

日期	版本	作者	修改说明
2022-06-16	V1.0.0	WJL	初始版本。
2022-06-20	V1.8.0	WJL	版本同步至 V1.8.0。
2022-09-20	V1.8.1	LinJianHua	支持Linux5.10 SDK
2022-11-20	V1.8.2	Caesar Wang	更新刷机说明
2023-04-20	V1.9.1	WJL	更新编译说明
2023-09-20	V2.0.0	WJL	更新说明
2023-12-20	V2.1.0	WJL	更新说明

目录

Rockchip RK3326 Linux SDK 快速入门

1. SDK预编译镜像
2. 开发环境搭建
 - 2.1 准备开发环境
 - 2.2 安装库和工具集
 - 2.2.1 检查和升级主机的 `python` 版本
 - 2.2.2 检查和升级主机的 `make` 版本
 - 2.2.3 检查和升级主机的 `lz4` 版本
 - 2.2.4 检查和升级主机的 `git` 版本
3. Docker 环境搭建
4. 软件开发指南
 - 4.1 开发向导
 - 4.2 芯片资料
 - 4.3 Buildroot开发指南
 - 4.4 第三方 OS 移植
 - 4.5 软件更新记录
5. 硬件开发指南
6. SDK 配置框架说明
 - 6.1 SDK 工程目录介绍
7. SDK 编译说明
 - 7.1 一键编译
 - 7.2 SDK 编译命令查看
 - 7.3 SDK 板级配置
 - 7.4 SDK 定制化配置
 - 7.5 全自动编译
 - 7.6 各模块编译
 - 7.6.1 U-Boot 编译
 - 7.6.2 Kernel 编译
 - 7.6.3 Recovery 编译
 - 7.6.4 Buildroot 编译
 - 7.6.5 交叉编译
 - 7.6.5.1 SDK 内置的交叉编译链
 - 7.6.5.2 Buildroot 的交叉编译
 - 7.6.6 固件的打包
8. 刷机说明
 - 8.1 Windows 刷机说明
 - 8.2 Linux 刷机说明
 - 8.3 系统分区说明

1. SDK预编译镜像

使用 RK3326 Linux SDK 预编译镜像，开发人员可以省去从源代码编译整个操作系统的过程，直接将预编译的镜像刷入 RK3326 开发板，从而快速启动开发和进行相关评估、对比，可减少因编译问题导致的开发时间和成本浪费。

可从公开地址下载 [SDK固件下载点击这里](#)

固件路径：通用 Linux SDK 固件-> Linux5.10 -> RK3326

如果需要修改SDK代码或快速入门，请参考下面章节。

2. 开发环境搭建

2.1 准备开发环境

我们推荐使用 Ubuntu 22.04 或更高版本的系统进行编译。其他的 Linux 版本可能需要对软件包做相应调整。除了系统要求外，还有其他软硬件方面的要求。

硬件要求：64 位系统，硬盘空间大于 40G。如果您进行多个构建，将需要更大的硬盘空间。

软件要求：Ubuntu 22.04 或更高版本系统。

2.2 安装库和工具集

使用命令行进行设备开发时，可以通过以下步骤安装编译 SDK 需要的库和工具。

使用如下 apt-get 命令安装后续操作所需的库和工具：

```
sudo apt-get update && sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \
libmpc-dev bc python-is-python3 python2
```

说明：

安装命令适用于 Ubuntu 22.04，其他版本请根据安装包名称采用对应的安装命令，若编译遇到报错，可以视报错信息，安装对应的软件包。其中：

- python 要求安装 python 3.6 或以上版本，此处以 python 3.6 为例。
- make 要求安装 make 4.0 或以上版本，此处以 make 4.2 为例。
- lz4 要求安装 lz4 1.7.3 或以上版本。
- git 没有 CVE-2022-39253 安全检测补丁。

2.2.1 检查和升级主机的 `python` 版本

检查和升级主机的 `python` 版本方法如下：

- 检查主机 `python` 版本

```
$ python3 --version
Python 3.10.6
```

如果不满足 `python>=3.6` 版本的要求， 可通过如下方式升级：

- 升级 `python 3.6.15` 新版本

```
PYTHON3_VER=3.6.15
echo "wget
https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

2.2.2 检查和升级主机的 `make` 版本

检查和升级主机的 `make` 版本方法如下：

- 检查主机 `make` 版本

```
$ make -v
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- 升级 `make 4.2` 新版本

```
$ sudo apt update && sudo apt install -y autoconf autopoint

git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDROOT_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

2.2.3 检查和升级主机的 `lz4` 版本

检查和升级主机的 `lz4` 版本方法如下：

- 检查主机 `lz4` 版本

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
```

- 升级 `lz4` 新版本

```
git clone https://gitee.com/mirrors/LZ4_old1.git
cd LZ4_old1

make
sudo make install
sudo install -m 0755 lz4 /usr/bin/lz4
```

2.2.4 检查和升级主机的 `git` 版本

- 检查主机 `git` 版本

```
$ git -v
git version 2.38.0
```

- 升级 `git` 新版本

```
$ sudo apt update && sudo apt install -y libcurl4-gnutls-dev

git clone https://gitee.com/mirrors/git.git --depth 1 -b v2.38.0
cd git
make git -j8
make install
sudo install -m 0755 git /usr/bin/git
```

3. Docker 环境搭建

为帮助开发者快速完成上面复杂的开发环境准备工作，我们也提供了交叉编译器 Docker 镜像，以便客户可以快速验证，从而缩短编译环境的构建时间。

使用 Docker 环境前，可参考如下文档进行操作

[<SDK>/docs/cn/Linux/Docker/Rockchip_Developer_Guide_Linux_Docker_Deploy_CN.pdf](https://docs.cn/Linux/Docker/Rockchip_Developer_Guide_Linux_Docker_Deploy_CN.pdf)。

已验证的系统如下：

发行版本	Docker 版本	镜像加载	固件编译
ubuntu 22.04	24.0.5	pass	pass
ubuntu 21.10	20.10.12	pass	pass
ubuntu 21.04	20.10.7	pass	pass
ubuntu 18.04	20.10.7	pass	pass
fedora35	20.10.12	pass	NR (not run)

Docker 镜像可从网址 [Docker镜像](#) 获取。

4. 软件开发指南

4.1 开发向导

为帮助开发工程师更快上手熟悉 SDK 的开发调试工作，随 SDK 发布

《Rockchip_Developer_Guide_Linux_Software_CN.pdf》，可在 `docs/cn/RK3326` 下获取，并会不断完善更新。

4.2 芯片资料

为帮助开发工程师更快上手熟悉 RK3326 的开发调试工作，随 SDK 发布《Rockchip RK3326-S Datasheet V1.1-20220301.pdf》芯片手册。可在 `docs/cn/RK3326/Datasheet` 下获取，并会不断完善更新。

4.3 Buildroot开发指南

为帮助开发工程师更快上手熟悉 Buildroot 系统的开发调试，随 SDK 发布

《Rockchip_Developer_Guide_Buildroot_CN.pdf》开发指南，可在 `docs/cn/Linux/System` 目录下获取，并会不断完善更新。

4.4 第三方 OS 移植

为帮助开发工程师更快上手熟悉第三方 OS 的移植适配，随 SDK 发布

《Rockchip_Developer_Guide_Third_Party_System_Adaptation_CN.pdf》开发向导，可在 `docs/cn/Linux/System` 目录下获取，并会不断完善更新。

4.5 软件更新记录

- 软件发布版本升级通过工程 XML 进行查看，具体方法如下：

```
# Linux4.4 SDK
.repo/manifests$ realpath rk3326_linux_release.xml
# 例如：打印的版本号为 v1.8.0，更新时间为 20220620
<SDK>/.repo/manifests/rk3326_linux_release_v1.8.0_20220620.xml

# Linux4.19 SDK
.repo/manifests$ realpath rk3326_linux4.19_release.xml
# 例如：打印的版本号为 v1.2.2，更新时间为 20221220
<SDK>/.repo/manifests/rk3326_linux4.19_release_v1.2.2_20221220.xml

# Linux5.10 SDK
.repo/manifests$ realpath rk3326_linux5.10_release.xml
# 例如： 打印的版本号为 v1.4.0，更新时间为 20231220
```

```
<SDK>/repo/manifests/rk3326_linux5.10_release_v1.4.0_20231220.xml
```

- 软件发布版本更新内容通过工程文本可以查看，具体方法如下：

```
# Linux4.4 SDK
<SDK>/repo/manifests/RK3326_Linux_SDK_Release_Note.md
或者
<SDK>/docs/cn/RK3326/RK3326_Linux_SDK_Release_Note.md

# Linux4.19 SDK
<SDK>/repo/manifests/RK3326_Linux4.19_SDK_Release_Note.md
或者
<SDK>/docs/cn/RK3326/RK3326_Linux4.19_SDK_Release_Note.md

# Linux5.10 SDK
<SDK>/repo/manifests/RK3326_Linux5.10_SDK_Release_Note.md
或者
<SDK>/docs/cn/RK3326/RK3326_Linux5.10_SDK_Release_Note.md
```

- 板端可通过如下方式获取版本信息

```
buildroot:/# cat /etc/os-release
NAME=Buildroot
VERSION=linux-5.10-gen-rkr7
ID=buildroot
VERSION_ID=2021.11
PRETTY_NAME="Buildroot 2021.11"
OS="buildroot"
BUILD_INFO="Tue Dec 26 16:39:24 CST 2023 - rockchip_rk3326_64"
KERNEL="5.10 - px30_linux_defconfig"
```

5. 硬件开发指南

硬件相关开发可以参考用户使用指南，在工程目录：

RK3326 硬件设计指南：

```
<SDK>/docs/cn/RK3326/Hardware/Rockchip_RK3326_Hardware_Design_Guide_V1.1_CN.pdf
```

RK3326 EVB 硬件开发指南：

```
<SDK>/docs/cn/RK3326/Hardware/Rockchip_RK3326_User_Manual_EVB_V1.0_CN.pdf
```

6. SDK 配置框架说明

6.1 SDK 工程目录介绍

SDK 目录包含有 app、buildroot、device、docs、external、kernel、output、prebuilts、rkdir、tools、u-boot 等目录。采用 manifest 来管理仓库，用 repo 工具来管理每个目录或其子目录会对应的 git 工程。

- app：存放上层应用 APP，主要是一些应用 Demo。
- buildroot：基于 Buildroot（2021.11）开发的根文件系统。
- device/rockchip：存放各芯片板级配置以及 SDK 编译和打包固件的脚本和文件等。
- docs：存放通用开发指导文件、芯片平台相关文档、Linux 系统开发相关文档、其他参考文档等。
- external：存放第三方相关仓库，包括显示、音视频、摄像头、网络、安全等。
- kernel：存放 Kernel 开发的代码。
- output：存放每次生成的固件信息、编译信息、XML、主机环境等。
- prebuilts：存放交叉编译工具链。
- rkdir：存放 Rockchip 相关二进制和工具。
- rockdev：存放编译输出固件，实际软链接到 `output/firmware`。
- tools：存放 Linux 和 Window 操作系统下常用工具。
- u-boot：存放基于 v2017.09 版本进行开发的 U-Boot 代码。

7. SDK 编译说明

SDK 可通过 `make` 或 `./build.sh` 加目标参数进行相关功能的配置和编译。

具体参考 `device/rockchip/common/README.md` 编译说明。

为了确保 SDK 每次更新都能顺利进行，建议在更新前清理之前的编译产物。这样做可以避免潜在的兼容性问题或编译错误，因为旧的编译产物可能不适用于新版本的 SDK。要清理这些编译产物，可以直接运行命令 `./build.sh cleanall`。

7.1 一键编译

芯片	类型	参考机型	一键编译	rootfs 编译	kernel 编译	uboot 编译
RK3326	开发板	RK3326 EVB LP3 V10	./build.sh lunch:rockchip_rk3326_evb_lp3_v10_32bit_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 px30_linux_defconfig rk3326_linux.config make ARCH=arm64 rk3326-evb-lp3-v10-linux.img	./make.sh rk3326
RK3326	开发板	RK3326 EVB LP3 V10	./build.sh lunch:rockchip_rk3326_evb_lp3_v10_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 px30_linux_defconfig rk3326_linux.config make ARCH=arm64 rk3326-evb-lp3-v10-linux.img	./make.sh rk3326
RK3326	开发板	RK3326 EVB LP3 V12	./build.sh lunch:rockchip_rk3326_evb_lp3_v12_32bit_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 px30_linux_defconfig rk3326_linux.config make ARCH=arm64 rk3326-evb-lp3-v12-linux.img	./make.sh rk3326
RK3326	开发板	RK3326 EVB LP3 V12	./build.sh lunch:rockchip_rk3326_evb_lp3_v12_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 px30_linux_defconfig rk3326_linux.config make ARCH=arm64 rk3326-evb-lp3-v12-linux.img	./make.sh rk3326
RK3326	开发板	RK3326 EVB LP3 V10	./build.sh lunch:rockchip_rk3326_robot64_defconfig&& ./build.sh	./build.sh rootfs	make ARCH=arm64 rk3326_linux_robot_defconfig make ARCH=arm64 rk3326-evb-lp3-v10-robot-linux.img	./make.sh rk3326
RK3326	开发板	RK3326 EVB LP3 V10	./build.sh lunch:rockchip_rk3326_robot64_no_gpu_defconfig&& ./build.sh	./build.sh rootfs	make ARCH=arm64 rk3326_linux_robot_defconfig make ARCH=arm64 rk3326-evb-lp3-v10-robot-no-gpu-linux.img	./make.sh rk3326

注意：

- **Kernel、U-boot 编译：**其中需要制指定工具链。

比如 SDK 内置 32 位工具链：

```
export CROSS_COMPILE=./prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin/arm-none-linux-gnueabi-
```

比如 SDK 内置 64 位工具链：

```
export CROSS_COMPILE=./prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
```

7.2 SDK 编译命令查看

`make help`，例如：

```
$ make help
menuconfig          - interactive curses-based configurator
oldconfig           - resolve any unresolved symbols in .config
synconconfig        - Same as oldconfig, but quietly, additionally update
deps
olddefconfig        - Same as synconconfig but sets new symbols to their
default value
savedefconfig       - Save current config to RK_DEFCONFIG (minimal config)
...
```

`make` 实际运行的是 `./build.sh`，

即可运行 `./build.sh <target>` 来编译相关功能，可通过 `./build.sh help` 查看具体编译命令。

```

$ ./build.sh -h

##### Rockchip Linux SDK #####

...

Usage: build.sh [OPTIONS]
Available options:
chip[:<chip>[:<config>]]      choose chip
defconfig[:<config>]         choose defconfig
config                        modify SDK defconfig
...
updateimg                    build update image
otapackage                    build A/B OTA update image
all                           build all images
release                       release images and build info
save                          save images and build info
all-release                   build and release images
allsave                       build all images and save them
shell                         setup a shell for developing
cleanall                      cleanup
clean[:module[:module]]...    cleanup modules
...
post-rootfs <rootfs dir>      trigger post-rootfs hook scripts
help                           usage

Default option is 'all'.

```

7.3 SDK 板级配置

进入工程 `<SDK>/device/rockchip/rk3326` 目录：

板级配置	说明
rockchip_defconfig	默认配置
rockchip_rk3326_evb_lp3_v10_32bit_defconfig rockchip_rk3326_evb_lp3_v10_defconfig	适用于 RK3326 EVB V10 搭配 LP3/LP4 开发板
rockchip_rk3326_evb_lp3_v12_32bit_defconfig rockchip_rk3326_evb_lp3_v12_defconfig	适用于 RK3326 EVB V12 搭配 LP3/LP4 开发板
rockchip_rk3326_robot64_defconfig rockchip_rk3326_robot64_no_gpu_defconfig	适用于 RK3326 Robot 小系统开发

可通过 `make lunch` 或者 `./build.sh lunch` 进行配置

```

$ ./build.sh lunch

##### Rockchip Linux SDK #####

...

Pick a defconfig:

```

```
1. rockchip_defconfig
2. rockchip_rk3326_evb_lp3_v10_32bit_defconfig
3. rockchip_rk3326_evb_lp3_v10_defconfig
4. rockchip_rk3326_evb_lp3_v12_32bit_defconfig
5. rockchip_rk3326_evb_lp3_v12_defconfig
6. rockchip_rk3326_robot64_defconfig
7. rockchip_rk3326_robot64_no_gpu_defconfig
Which would you like? [1]:
...
```

其他功能的配置可通过 `make menuconfig` 来配置相关属性。

7.4 SDK 定制化配置

SDK 可通过 `make menuconfig` 进行相关配置，目前可配组件主要如下：

```
(rk3326) SoC
  Rootfs --->
  Loader (u-boot) --->
  Kernel --->
  Boot --->
  Recovery (based on buildroot) --->
  PCBA test (based on buildroot) --->
  Security --->
  Extra partitions --->
  Firmware --->
  Update (Rockchip update image) --->
  Others configurations --->
```

- Rootfs: 这里的 Rootfs 代表“根文件系统”，在这里可选择 Buildroot 等不同的根文件系统配置。
- Loader (u-boot): 这是引导加载器的配置，通常是 u-boot，用于初始化硬件并加载主操作系统。
- Kernel: 这里配置内核选项，定制适合自己的硬件和应用需求的 Linux 内核。
- Boot: 这里配置 Boot 分区支持格式。
- Recovery (based on buildroot): 这是基于 buildroot 的 recovery 环境的配置，用于系统恢复和升级。
- PCBA test (based on buildroot): 这是一个基于 buildroot 的 PCBA（印刷电路板组装）测试环境的配置。
- Security: 安全功能开启，包含 Secureboot 开启方式、Optee 存储方式、烧写 Key 等。
- Extra partitions: 用于配置额外的分区。
- Firmware: 在这里配置固件相关选项。
- Update (Rockchip update image): 用于配置 Rockchip 完整固件的选项。
- Others configurations: 其他额外的配置选项。

通过 `make menuconfig` 配置界面提供了一个基于文本的用户界面来选择和配置各种选项。

配置完成后，使用 `make savedefconfig` 命令保存这些配置，这样定制化编译就会根据这些设置来进行。

通过以上 config，可选择不同 Rootfs/Loader/Kernel 等配置，进行各种定制化编译，这样就可以灵活地选择和配置系统组件以满足特定的需求。

7.5 全自动编译

为了确保软件开发套件（SDK）的每次更新都能顺利进行，建议在更新前清理之前的编译产物。这样做可以避免潜在的兼容性问题或编译错误，因为旧的编译产物可能不适用于新版本的 SDK。要清理这些编译产物，可以直接运行命令 `./build.sh cleanall`。

进入工程根目录执行以下命令自动完成所有的编译：

```
./build.sh all # 只编译模块代码 (u-Boot, kernel, Rootfs, Recovery)
               # 需要再执行`./build.sh ./mkfirmware.sh`进行固件打包

./build.sh     # 编译模块代码 (u-Boot, kernel, Rootfs, Recovery)
               # 打包成 update.img 完整升级包
               # 所有编译信息复制和生成到 out 目录下
```

7.6 各模块编译

7.6.1 U-Boot 编译

```
./build.sh uboot
```

7.6.2 Kernel 编译

- 方法一

```
./build.sh kernel
```

- 方法二

```
cd kernel
export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-
x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
make ARCH=arm64 px30_linux_defconfig rk3326_linux.config
make ARCH=arm64 rk3326-evb-lp3-v10-linux.img -j
或者
make ARCH=arm64 rk3326-evb-lp3-v12-linux.img -j

注意：`rk3326-evb-lp3-v10-linux`可替换具体的硬件 dts
```

- 方法三

```
cd kernel
export CROSS_COMPILE=aarch64-linux-gnu-
make ARCH=arm64 px30_linux_defconfig rk3326_linux.config
make ARCH=arm64 rk3326-evb-lp3-v10-linux.img -j
或者
make ARCH=arm64 rk3326-evb-lp3-v12-linux.img -j
```

注意：`rk3326-evb-lp3-v10-linux` 可替换具体的硬件 dts

7.6.3 Recovery 编译

进入工程根目录执行以下命令自动完成 Recovery 的编译及打包。

```
./build.sh recovery
```

编译后在 Buildroot 目录 `output/rockchip_rk3326_recovery/images` 生成 `recovery.img`。

注：recovery.img 是包含内核，所以每次 Kernel 更改，Recovery 是需要重新打包生成。Recovery 重新打包的方法如下：

```
source buildroot/envsetup.sh
cd buildroot
make recovery-reconfigure
cd -
./build.sh recovery
```

注：Recovery 是非必需的功能，有些板级配置不会设置

7.6.4 Buildroot 编译

进入工程目录根目录执行以下命令自动完成 Rootfs 的编译及打包：

```
./build.sh rootfs
```

编译后在 Buildroot 目录 `output/rockchip_rk3326_64/images` 下生成不同格式的镜像，默认使用 `rootfs.ext4` 格式。

具体可参考 Buildroot 开发文档参考：

```
<SDK>/docs/cn/Linux/System/Rockchip_Developer_Guide_Buildroot_CN.pdf
```

7.6.5 交叉编译

7.6.5.1 SDK 内置的交叉编译链

目录	说明
prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu	gcc arm 10.3.1 64位工具链
prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi	gcc arm 10.3.1 32位工具链

可从以下地址下载工具链：

[点击这里](#)

7.6.5.2 Buildroot 的交叉编译

可通过 `source buildroot/envsetup.sh` 来设置不同芯片和目标功能的配置

```
$ source buildroot/envsetup.sh
Top of tree: rk3326

Pick a board:

...
24. rockchip_rk3326_32
25. rockchip_rk3326_64
26. rockchip_rk3326_recovery
...

Which would you like? [1]:
```

默认选择 25，rockchip_rk3326_64。然后进入 RK3326 的 Buildroot 目录，开始相关模块的编译。

其中 `rockchip_rk3326_32` 是 32 位 Buildroot 系统编译，`rockchip_rk3326_recovery` 是用来编译 Recovery 模块。

比如编译 rockchip-test 模块，常用相关编译命令如下：

- 进入 buildroot 目录

```
SDK$ cd buildroot
```

- 删除并重新编译 rockchip-test

```
SDK$ make rockchip-test-recreate
```

- 重编 rockchip-test

```
SDK$ make rockchip-test-rebuild
```

- 删除 rockchip-test

```
SDK$ make rockchip-test-dirclean
或者
SDK$ rm -rf /buildroot/output/rockchip_rk3326_64/build/rockchip-test-master
```

若需要编译单个模块或者第三方应用，需交叉编译环境进行配置。比如 RK3326 其交叉编译工具位于 `buildroot/output/rockchip_rk3326_64/host/usr` 目录下，需要将工具的 `bin/` 目录和 `aarch64-buildroot-linux-gnu/bin/` 目录设为环境变量，在顶层目录执行自动配置环境变量的脚本：

```
source buildroot/envsetup.sh rockchip_rk3326_64
```

输入命令查看：

```
cd buildroot/output/rockchip_rk3326_64/host/usr/bin  
./aarch64-linux-gcc --version
```

此时会打印如下信息：

```
aarch64-linux-gcc.br_real (Buildroot) 12.3.0
```

保存到 rootfs 配置文件

```
buildroot$ make update-defconfig
```

7.6.6 固件的打包

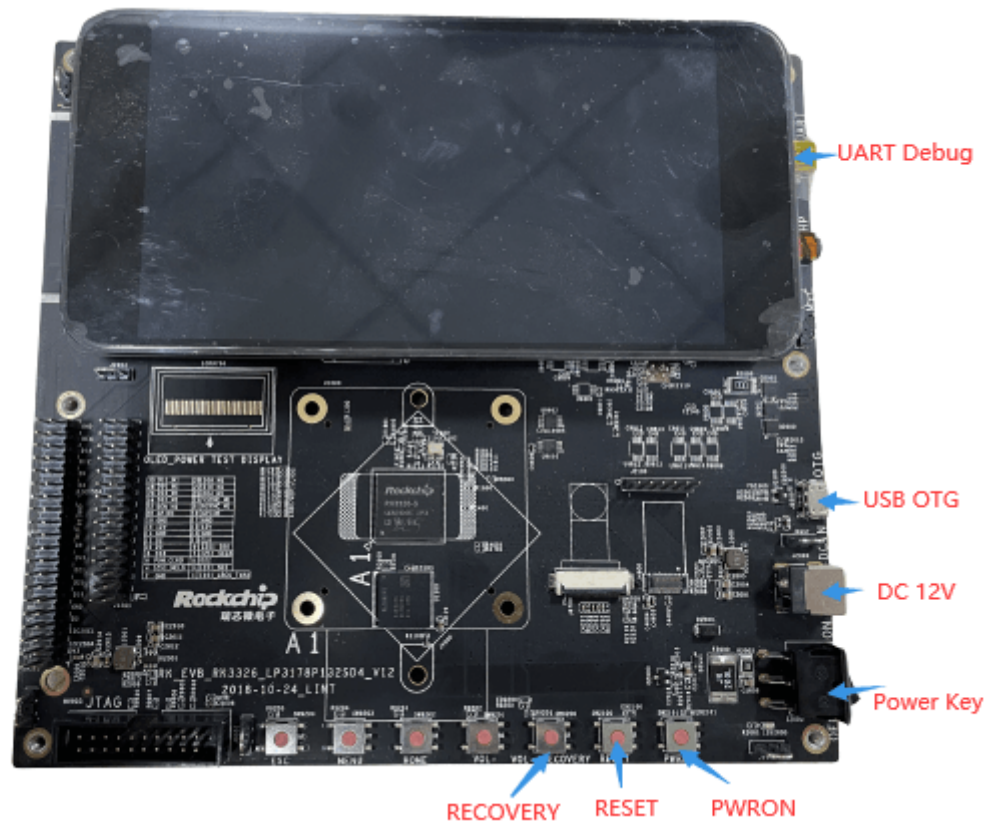
上面 Kernel、U-Boot、Recovery、Rootfs 各个部分的编译后，进入工程目录根目录执行以下命令自动完成所有固件打包到 `rockdev` 目录下：

固件生成：

```
./build.sh firmware
```

8. 刷机说明

RK3326-EVB 接口分布图如下：



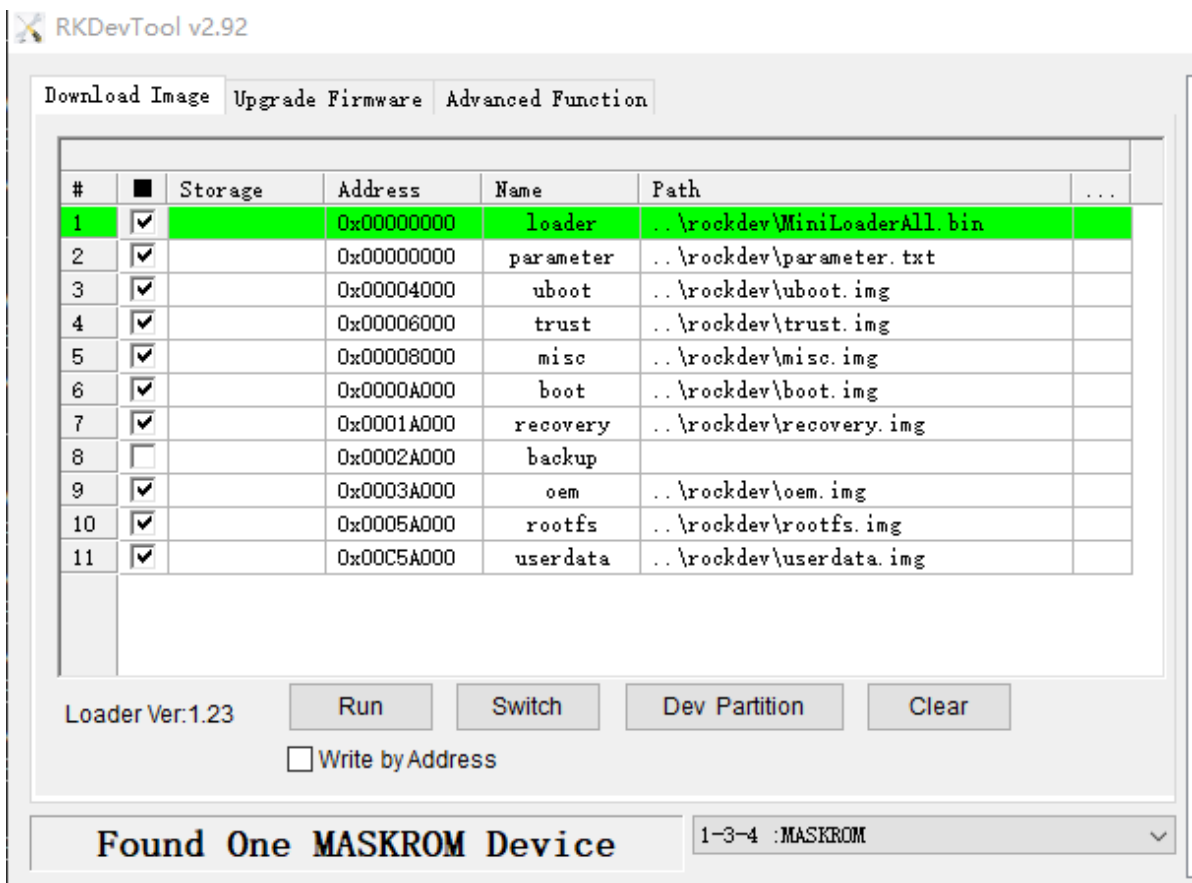
8.1 Windows 刷机说明

SDK 提供 Windows 烧写工具（工具版本需要 V2.92 或以上），工具位于工程根目录：

```
tools/windows/RKDevTool
```

如下图，编译生成相应的固件后，设备烧写需要进入 MASKROM 或 BootROM 烧写模式，连接好 USB 下载线后，按住按键“MASKROM”不放并按下复位键“RST”后松手，就能进入 MASKROM 模式，加载编译生成固件的相应路径后，点击“执行”进行烧写，也可以按“recovery”按键不放并按下复位键“RST”后松手进入 loader 模式进行烧写，下面是 MASKROM 模式的分区偏移及烧写文件。

（注意：Windows PC 需要在管理员权限运行工具才可执行）



注：烧写前，需安装最新 USB 驱动，驱动详见：

<SDK>/tools/windows/DriverAssitant_v5.12.zip

8.2 Linux 刷机说明

Linux 下的烧写工具位于 tools/linux 目录下（Linux_Upgrade_Tool 工具版本需要 V2.1 或以上），请确认你的板子连接到 MASKROM/loader rockusb。比如编译生成的固件在 rockdev 目录下，升级命令如下：

```

sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -t rockdev/trust.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rockdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd

```

或升级打包后的完整固件：

```

sudo ./upgrade_tool uf rockdev/update.img

```

或在根目录，机器在 MASKROM 状态运行如下升级：

```

./rkflash.sh

```

8.3 系统分区说明

默认分区说明（下面是 RK3326 EVB 分区参考）

Number	Start (sector)	End (sector)	Size	Name
1	16384	24575	4096K	uboot
2	24576	32767	4096K	trust
3	32768	40959	4096K	misc
4	40960	106495	32M	boot
5	106496	303104	32M	recovery
6	172032	237567	32M	bakcup
7	237568	368639	64M	oem
8	368640	12951551	6144M	rootfs
9	12951552	30535646	8585M	userdata

- uboot 分区：供 uboot 编译出来的 uboot.img。
- trust 分区：供 uboot 编译出来的 trust.img。
- misc 分区：供 misc.img，给 recovery 使用。
- boot 分区：供 kernel 编译出来的 boot.img。
- recovery 分区：供 recovery 编译出的 recovery.img。
- backup 分区：预留，暂时没有用，后续跟 Android 一样作为 recovery 的 backup 使用。
- oem 分区：给厂家使用，存放厂家的 APP 或数据。挂载在 /oem 目录。
- rootfs 分区：供 buildroot 编出来的 rootfs.img。
- userdata 分区：供 APP 临时生成文件或给最终用户使用，挂载在 /userdata 目录下。